

SpendGuard - POC

1. Problem Statement [🔗](#)

Managing personal finances can be overwhelming and time-consuming. Many budgeting apps require manual entry of expenses, which is a friction point for users. Additionally, visual analytics and complex features are often unnecessary for users who simply want to track their spending and stay within a budget. A simplified solution is needed where users can automatically monitor their financial activity without the complexity of advanced features or dashboards.

2. Objective [🔗](#)

The objective of this POC is to build a simple, intuitive budgeting app that:

- Automatically pulls transaction data from users' bank accounts using **Plaid API**.
- Tracks the user's **budget** against their **spending**.
- Notifies users when they reach predefined **spending thresholds** (e.g., 80% or 90% of their budget).
- Avoids complex visual analytics and focuses on minimalism.

3. Core Features for POC [🔗](#)

- **User Authentication:** Users can sign up or log in using a simple email/password flow (or OAuth).
- **Plaid Integration:** Users can link their bank account using **Plaid Link**, allowing the app to securely access their transaction data.
- **Budget Setting:** Users can define their monthly budget amount (e.g., \$1,000).
- **Transaction Syncing:** The app pulls transactions from Plaid every day to check the user's spending.
- **Threshold Alerts:** Notify users when they hit preset spending thresholds (e.g., 80%, 90% of their budget).
- **Minimal UI:** A simple interface that shows users their budget, spending, and current threshold status.

4. Non-Goals (for POC) [🔗](#)

- **Detailed Transaction Tracking:** The app will not display individual transactions or merchant details.
- **Visual Analytics/Dashboards:** No charts, graphs, or detailed financial analysis in the POC. The focus is purely on budget vs. spending tracking.
- **Advanced Features:** No support for features like bill payments, saving goals, or investment tracking.
- **Manual Expense Entry:** Users will not need to enter transactions manually — everything is automated through Plaid.

5. Target Users (Initial Audience) [🔗](#)

- **Age Group:** Primarily 18–45 years old.
- **Tech-Savvy Individuals:** Users comfortable with linking their bank accounts and using digital tools for personal finance.
- **Budget-Conscious Individuals:** Those who are looking to maintain control over their spending without needing a complex tool.
- **Non-Finance Experts:** Users who don't want to deal with complicated financial planning and prefer a simple, no-frills solution.

6. Tech Stack [🔗](#)

- **Frontend:**
 - **React:** For building the user interface.
 - **Tailwind CSS:** For fast, responsive, and minimal design.
 - **React Plaid Link:** To handle Plaid's bank account linking.
- **Backend:**

- **Node.js** with **Express**: For creating RESTful APIs.
- **MongoDB** (or **Firebase Firestore**): To store user data, budgets, transactions, and thresholds.
- **Plaid API**: To pull transaction data from the user's bank accounts.
- **JWT Authentication**: For secure login and user sessions.
- **Authentication**:
 - **Firebase Authentication**: For user sign-up/login.
- **Hosting**:
 - **Firebase Hosting** (or **Vercel/Netlify**): For hosting the frontend.
 - **Heroku** (or **AWS / DigitalOcean**) for backend hosting if necessary.

7. Architecture Overview [↗](#)

1. Frontend (React):

- The user interface will be a simple dashboard showing:
 - The user's budget.
 - The total amount spent.
 - A progress bar (or text) showing how much of the budget is used.
 - Alerts for when the user is approaching the threshold.
- **Plaid Link** will be used to allow the user to securely link their bank account.
- **Firebase Authentication** will manage user authentication.
- The app will make **API calls** to the backend to fetch user data (transactions, budgets) and check if any thresholds have been met.

2. Backend (Node.js/Express):

- Will handle user authentication (via **JWT**).
- Will interact with **Plaid API** to pull transaction data and store it in the database.
- Will calculate the percentage of the budget used, compare it against set thresholds, and notify the frontend if an alert should be triggered.

3. Database:

- User profiles will be stored in **MongoDB** (or **Firestore** if Firebase is chosen).
- Budget, spending, and alert thresholds will be stored alongside the user's data in the database.
- Transaction data pulled from **Plaid API** will be stored in a way that associates each transaction with the respective user.

4. Plaid API:

- Used to securely pull transaction data once the user links their bank account.
- The backend will make periodic calls to the Plaid API to fetch new transaction data.

5. Threshold Logic:

- The backend will calculate how much of the user's budget has been used and compare it against user-defined thresholds (80%, 90% of the budget).
- If a threshold is reached, an alert will be triggered and sent to the frontend.

8. Success Criteria [↗](#)

- **User Authentication**: Users can sign up, log in, and securely authenticate.
- **Plaid Integration**: Users can successfully link their bank accounts, and the app can pull transaction data.
- **Budget and Spending Tracking**: Users can set a budget, and the app will track spending and compare it with the budget automatically.
- **Threshold Alerts**: Users will receive notifications when they approach or exceed the preset thresholds (80%, 90%, etc.).
- **Simple UI**: The app should have a minimal design with only essential features.

- **Backend Stability:** The backend should properly sync transactions from Plaid and maintain accurate records of user spending.

9. Future Considerations [↗](#)

- **More Complex Threshold Logic:** Allow users to define multiple thresholds for different categories (e.g., "Food" vs. "Entertainment").
- **Recurring Transactions:** Handle recurring expenses (e.g., subscriptions) and subtract them from the budget automatically.
- **Detailed Transaction Data:** Allow users to view transaction details (merchant name, date, amount).
- **Multiple Accounts:** Allow users to link multiple bank accounts and aggregate spending from them.
- **Mobile App:** Develop a mobile version of the app for iOS/Android for on-the-go access to budget tracking.
- **Saving Goals:** Add features that let users track their savings goals alongside budgets.